

MOBILE NOTIFICATION PLATFORM

Ship push & in-app messaging with BNotify SDK

Production-ready SDKs for Android, iOS, and Flutter. Web and React Native are on the roadmap. Create a Console project first, then follow the platform guide.

[Create a project](#)[Android guide](#)[Flutter guide](#)[Download PDF](#)

Choose your platform

Start in the [Console](#) (project + app + config file), then follow the platform guide: repositories, initialization, permissions, and verification.

AND

Android

Add the Android SDK, initialize once, and receive system notifications plus in-app campaigns.

AVAILABLE

minSdk 24 · v1.0.5

iOS

iOS

Add the iOS SDK, enable push, and track clicks, dismissals, and optional rich-media notifications.

AVAILABLE

iOS 13+

WEB

Web

Browser push for websites. The Web platform already appears in Console; the SDK and this guide will ship when it is ready.

COMING SOON

[Notify me via Contact](#)

FL

Flutter

Add the Dart plugin from pub.dev. It wraps the native Android and iOS BNotify SDKs for push and in-app messaging.

AVAILABLE

bnotify_flutter 0.1.2

RN

React Native

Native module for Expo and bare React Native workflows. Integration docs will mirror Android / iOS capabilities.

COMING SOON

[Notify me via Contact](#)

What BNotify provides

Capability	Android	iOS
Device registration	Automatic after SDK init	Automatic after SDK init
System push UI	Tray notifications	Lock screen / banner notifications
Click / dismiss analytics	Handled by the SDK	Report via <code>trackEvent</code>
In-app messages	Modal, card, banner, and more	Modal, card, banner, and more
Expanded images / GIF	Supported	Supported (Notification Service Extension)

Important

The SDK does **not** send campaigns. Register the device with BNotify, then create and send messages from the BNotify dashboard, or from your own server with the [Backend API](#). The SDK displays notifications and reports engagement.

Quick links

- Create a project & add an app: [Console setup guide](#)
- BNotify Console Dashboard: console.bnotify.com (<https://console.bnotify.com/>)
- Android package: jitpack.io/#bnotify/bnotifysdk (<https://jitpack.io/#bnotify/bnotifysdk>)
- Android source: github.com/bnotify/bnotifysdk (<https://github.com/bnotify/bnotifysdk>)
- iOS SPM: gitlab.com/hamza.hafeez/bnotifyiossdk (<https://gitlab.com/hamza.hafeez/bnotifyiossdk>)
- Flutter pub.dev: pub.dev/packages/bnotify_flutter (https://pub.dev/packages/bnotify_flutter)
- Flutter guide: [Flutter integration](#)
- Send from your server: [Backend API](#)
- Offline copy: [BNotify SDK Documentation \(PDF\)](#)
- Support: [Contact](#) · [FAQ](#)

© bnotify.com · Mobile Notification Platform

Docs version 1.0 · Platforms: Android, iOS, Flutter (Web & RN coming soon)

BNotify SDK Documentation · <https://bnotify.github.io/sdk-docs/>

GETTING STARTED

Create a project and add an app

Before you add the SDK to your code, set up a project in the BNotify Console, register your app, and download the config file. Then follow the platform integration guide.

Console console.bnotify.com

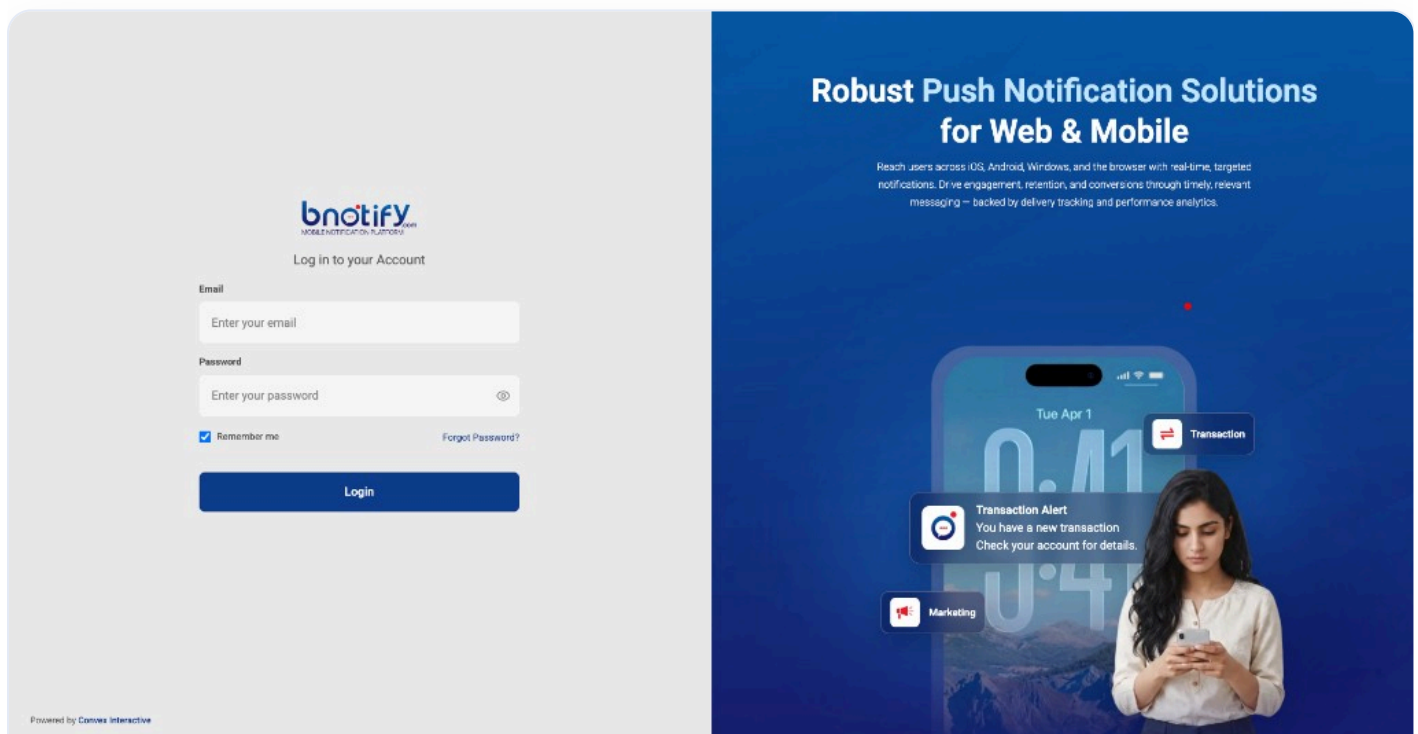
Example Android app

Order of work

Console first (this page) → download the config file → then [Android](#) or [iOS](#) SDK integration.

Step 1 — Open the Console and log in

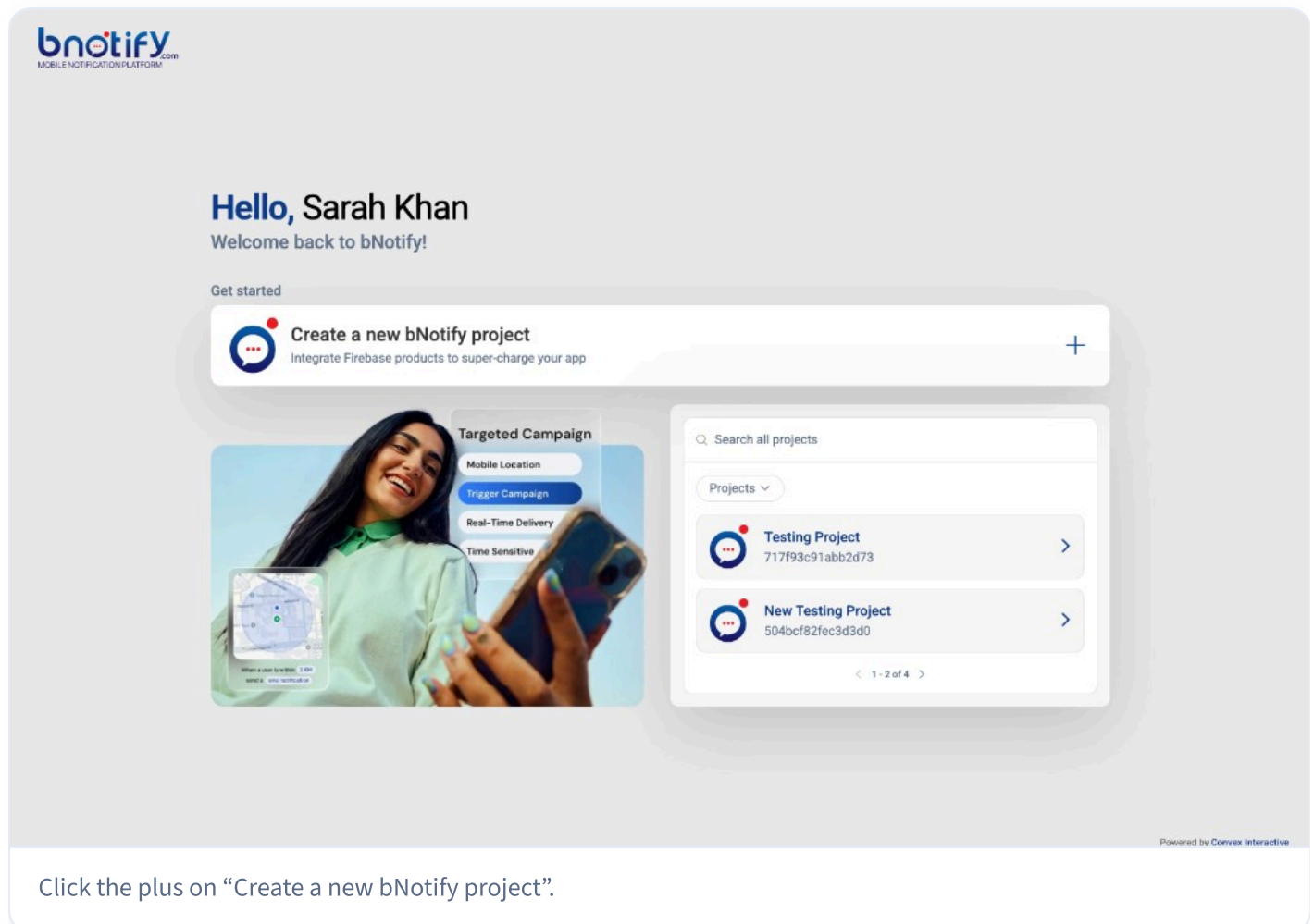
Open <https://console.bnotify.com/> (<https://console.bnotify.com/>). Enter your email and password, then click **Login**.



Log in to the BNotify Console.

Step 2 — Create a new project

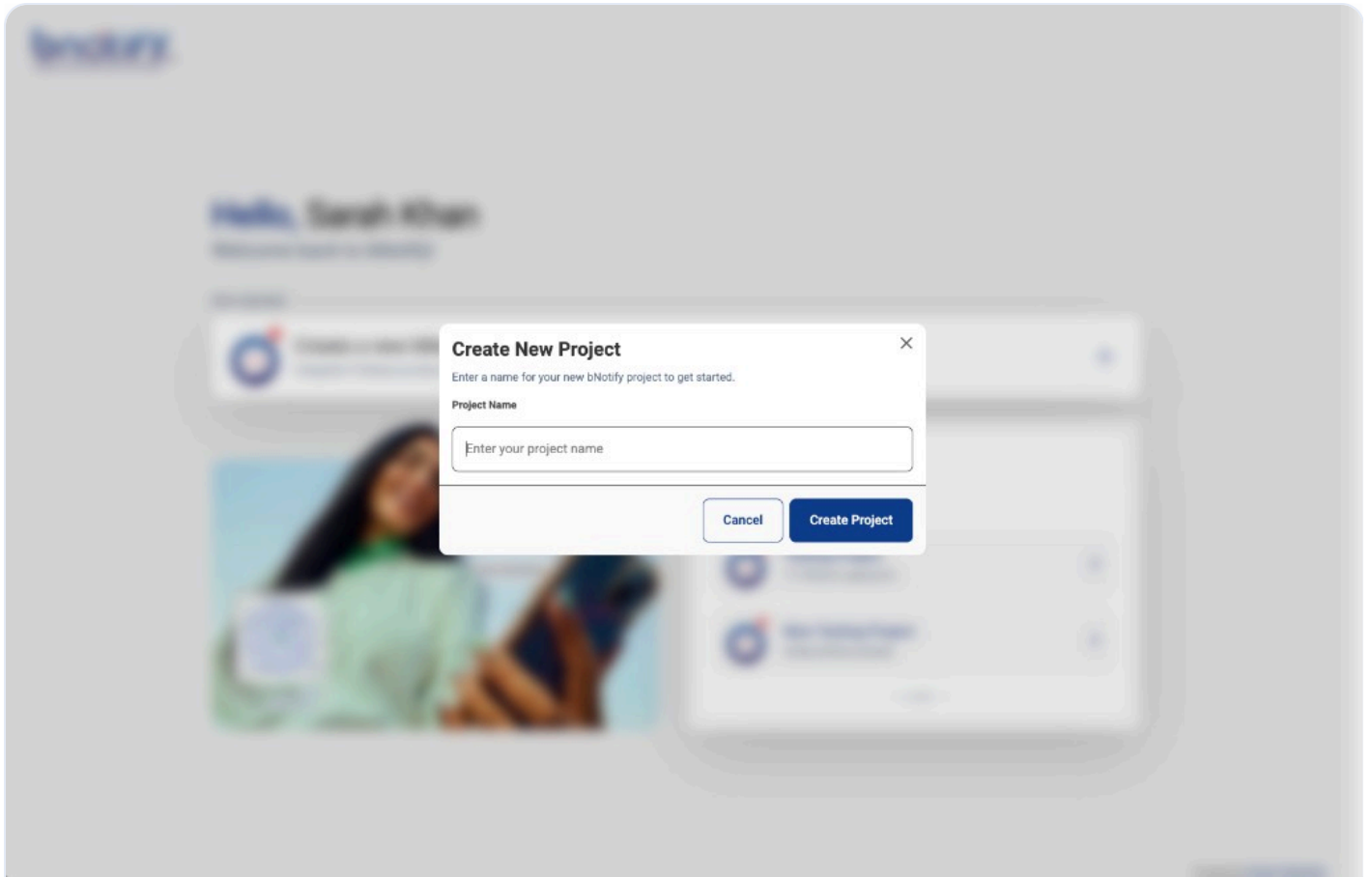
After login you land on the projects home. Click **Create a new bNotify project** (the plus control on the card).



The screenshot shows the bNotify dashboard. At the top left is the bnotify.com logo with the tagline 'MOBILE NOTIFICATION PLATFORM'. Below the logo, it says 'Hello, Sarah Khan' and 'Welcome back to bNotify!'. A 'Get started' section contains a card titled 'Create a new bNotify project' with the subtitle 'Integrate Firebase products to super-charge your app' and a plus icon. Below this card is a 'Targeted Campaign' section featuring a woman holding a smartphone and a map. To the right is a 'Projects' list with a search bar and two entries: 'Testing Project' (ID: 717f93c91abb2d73) and 'New Testing Project' (ID: 504bcf82fec3d3d0). A footer note at the bottom right states 'Powered by Convex Interactive'.

Click the plus on “Create a new bNotify project”.

In the dialog, enter a **Project Name** and click **Create Project**.

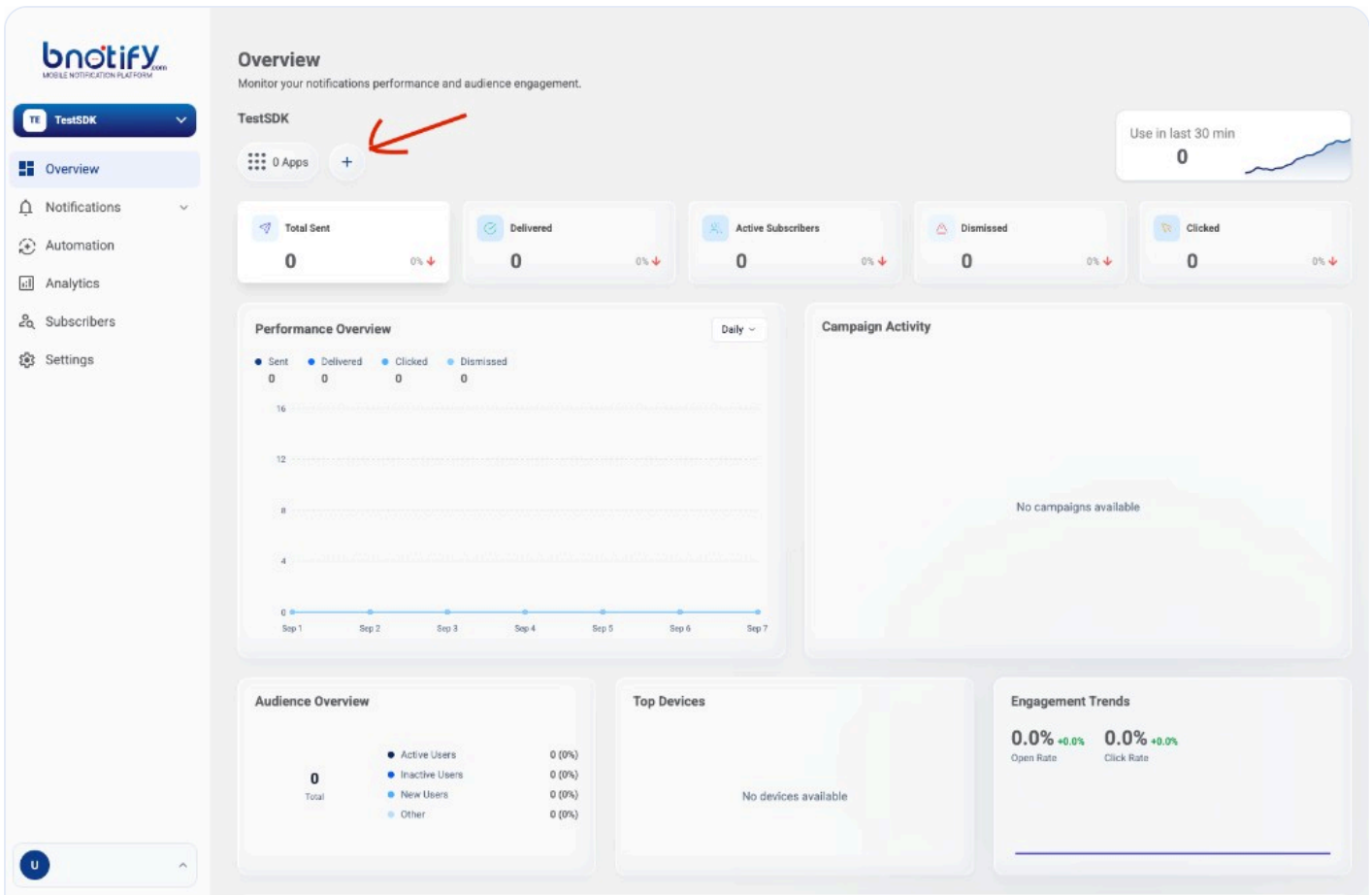


Enter a project name, then Create Project.

The project **Dashboard / Overview** opens as soon as the project is created.

Step 3 — Add an app

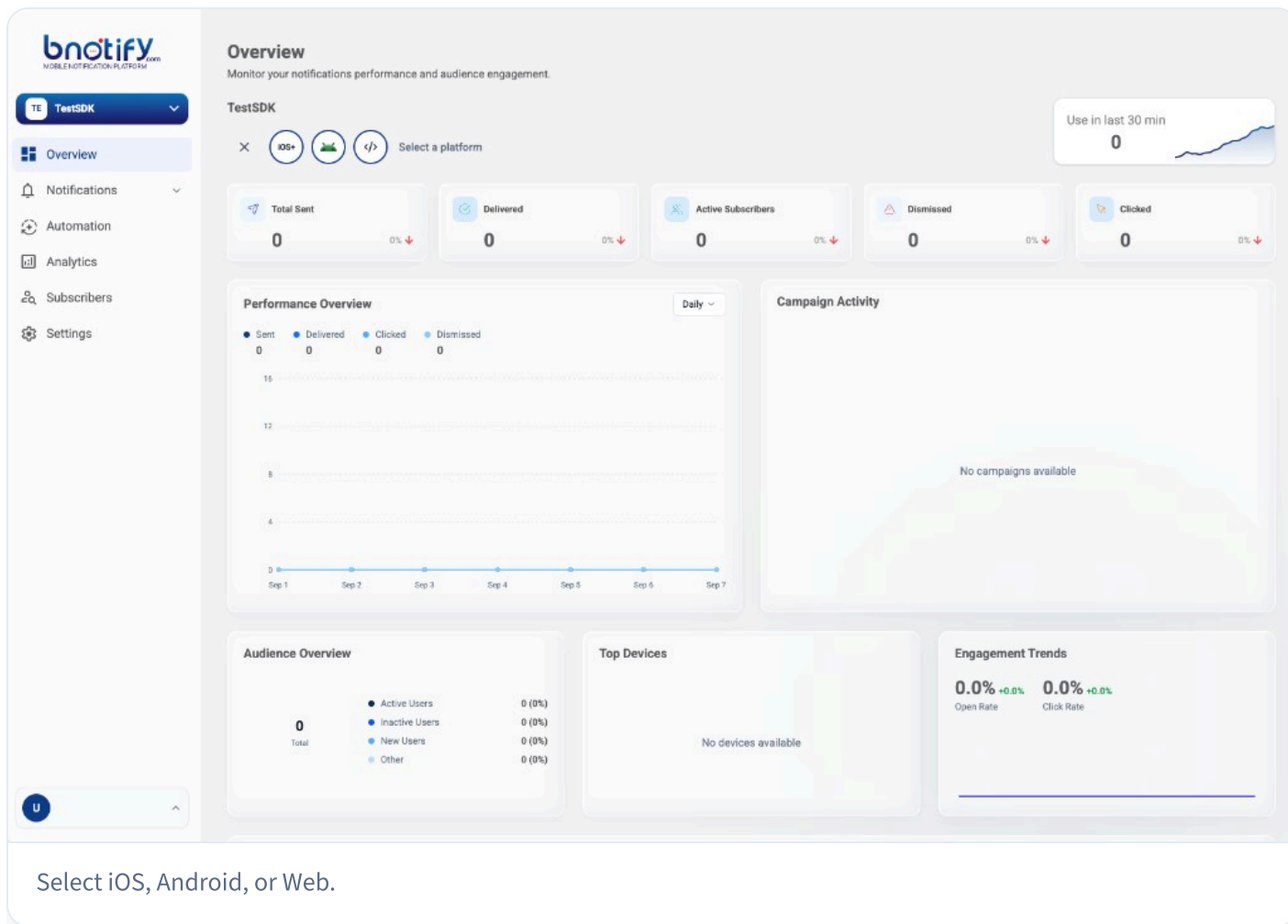
On Overview, a new project shows **0 Apps**. Click the circular **Plus (+)** button next to it.



Click Plus (+) beside 0 Apps.

You will see platform options:

- **iOS** — live SDK
- **Android** — live SDK (used in this example)
- **Web** — coming soon



For this walkthrough, choose **Android**. That opens the Add App wizard.

Field	What to enter
Android package name	Your app's package, e.g. <code>com.company.appname</code>
App nickname	A label for this app in the Console
FCM Project ID	From your Firebase / FCM project settings
FCM App ID	From your Firebase / FCM app settings
FCM API Key	From your Firebase / FCM project settings
FCM Sender ID	From your Firebase / FCM project settings
FCM private key (.json)	Upload the service-account JSON from Firebase

Click **Register app**.

×
Add bNotify to your Android app
Go to docs

1

Register app

Android package name *

com.company.appname

App nickname *

My Android App

FCM Project ID *

FCM App ID *

FCM API Key *

FCM Sender ID *

FCM private key (.json) *

Choose file No file chosen

Register app

2

Download and then add config file

3

Add Firebase SDK

4

Next steps

Register app: package name, nickname, and FCM fields.

Step 5 — Download the BNotify JSON file

After registration, the wizard asks you to **download the BNotify JSON config file**. Download it and keep it — you will add it to your Android (or Flutter) project as `app/bnotify-config.json`.

Step 6 — Finish the remaining wizard steps

Click **Next** through the remaining Console steps until the wizard completes. Your Android application is then created and configured in the BNotify Console.

Then integrate the SDK

Place the downloaded JSON in your app, then follow the [Android integration guide](#), the [iOS guide](#), or the [Flutter guide](#) if you are building a Flutter app (you still register both Android and iOS apps in Console).

iOS, Flutter, and Web

The same flow applies: Plus (+) → pick a platform.

- **iOS** — register the app in Console, download `bnotify-config.plist`, then follow [iOS integration](#).
- **Flutter** — register Android and iOS apps, download both config files, then follow [Flutter integration](#) (`bnotify_flutter` on pub.dev).
- **Web** — listed in Console; the Web SDK is [coming soon](#).

Next: [Android SDK integration](#) →

[Open Console](#)

Android SDK integration

Add BNotify to any Android app with JitPack, the Gradle config plugin, and a few lifecycle hooks. After setup, devices register automatically and can receive pushes plus in-app campaigns.

Create the app in Console first

Before Gradle setup, [create a project](#), add an Android app, and download `bnotify-config.json` . Place that file at `app/bnotify-config.json` , then continue below.

Artifact `bnotifysdk + bnotifyplugin`

Version `1.0.5`

minSdk `24`

Source [GitHub](#)

What you get after integration

Capability	Result
Device registration	Device appears in the BNotify dashboard after init
Push receive	System tray notifications
System notification UI	Shown automatically by the SDK (default)
Click / dismiss analytics	Reported to BNotify
Token callback	<code>OnTokenListener.onNewToken</code>
In-app campaigns	Shown in-app when a campaign is due

Dashboard delivers campaigns

The Android SDK does not send notifications. Create and send campaigns from the BNotify dashboard. The SDK registers the device, displays messages, and reports engagement.

Prerequisites

Requirement	Notes
Android Studio + Kotlin DSL	Examples use <code>.kts</code>
<code>minSdk</code> ≥ 24	Matches SDK
Internet on device	Required for registration and delivery
BNotify credentials	Download <code>bnotify-config.json</code> after you register the Android app in Console
Config fields	Use the values provided in the dashboard download
Physical device	Recommended for notification testing

Integration checklist

- 1 Add JitPack to `settings.gradle.kts`
- 2 Add plugin classpath in project `build.gradle.kts`
- 3 `implementation` SDK + `apply` plugin in app module
- 4 Add `app/bnotify-config.json` (downloaded from Console)
- 5 Sync Gradle → confirm `GeneratedConfig`
- 6 Call `BNotifyApp.initializeBase` in `Application`
- 7 Register custom `Application` in manifest
- 8 `handleIntent` + token listener in launcher Activity

9 Subclass `BNotifyMessagingService` (recommended)

10 Handle `screen` extras for multi-screen routing

Step 1 — Repositories (`settings.gradle.kts`)

Add JitPack so Gradle can resolve the SDK and plugin. Merge into existing blocks if present.

KOTLIN

```
pluginManagement {
    repositories {
        google {
            content {
                includeGroupByRegex("com\\.\\.android.*")
                includeGroupByRegex("com\\.\\.google.*")
                includeGroupByRegex("androidx.*")
            }
        }
        mavenCentral()
        gradlePluginPortal()
        maven { url = uri("https://jitpack.io") }
    }
}

dependencyResolutionManagement {
    repositoriesMode.set(RepositoriesMode.FAIL_ON_PROJECT_REPOS)
    repositories {
        google()
        mavenCentral()
        maven { url = uri("https://jitpack.io") }
    }
}
```

Step 2 — Plugin classpath (project `build.gradle.kts`)

KOTLIN

```
plugins {  
    alias(libs.plugins.android.application) apply false  
    alias(libs.plugins.kotlin.android) apply false  
}  
  
buildscript {  
    repositories {  
        maven(url = "https://jitpack.io")  
    }  
    dependencies {  
        classpath("com.github.bnotify.bnotifysdk:bnotifyplugin:1.0.5")  
    }  
}
```

Coordinates

Library: `com.github.bnotify.bnotifysdk:bnotifysdk:1.0.5`

Plugin apply id: `com.github.bnotifysdk.bnotifyplugin`

Confirm on [JitPack](https://jitpack.io/#bnotify/bnotifysdk) (<https://jitpack.io/#bnotify/bnotifysdk>).

Step 3 — Dependency + plugin (app module)

KOTLIN

```
plugins {  
    alias(libs.plugins.android.application)  
    alias(libs.plugins.kotlin.android)  
}  
  
android {  
    namespace = "com.example.yourapp"  
    compileSdk = 36  
    defaultConfig {  
        applicationId = "com.example.yourapp"  
        minSdk = 24  
        targetSdk = 36  
    }  
}  
  
dependencies {  
    implementation("com.github.bnotifysdk:bnotifysdk:1.0.5")  
}  
  
apply(plugin = "com.github.bnotifysdk.bnotifyplugin")
```

On sync, the plugin reads `app/bnotify-config.json` and generates `GeneratedConfig.kt` under your `applicationId` package. Missing JSON fails the build with a clear error.

Step 4 — Add `app/bnotify-config.json`

Download this file from the Console after you register the Android app (see [Create a project](#)). Path required by the plugin: `{projectRoot}/app/bnotify-config.json`.

JSON

```
{
  "projectId": "your-project-id",
  "packageName": "com.example.yourapp",
  "apiKey": "your-api-key",
  "authDomain": "",
  "databaseURL": "",
  "storageBucket": "",
  "messagingSenderId": "",
  "appId": "your-app-id",
  "measurementId": "",
  "fcmAppId": "1:1234567890:android:abcdef",
  "fcmProjectId": "your-firebase-project-id",
  "fcmApiKey": "AIzaSy...",
  "fcmSenderId": "1234567890"
}
```

Field	Required	Used for
<code>projectId</code> , <code>appId</code> , <code>apiKey</code> , <code>packageName</code>	Yes	Project / app identity
<code>fcmAppId</code> , <code>fcmProjectId</code> , <code>fcmApiKey</code> , <code>fcmSenderId</code>	Yes	Included in the dashboard config file
Other fields	Optional	Reserved / future

Step 5 — Application class

KOTLIN

```
class MyApplication : Application() {
    override fun onCreate() {
        super.onCreate()
        BNotifyApp.initializeBase(
            this,
            MainActivity::class.java,
            GeneratedConfig.JSON.toString()
        )
    }
}
```

Call this once in `Application.onCreate`. It registers the device, enables notifications, and starts in-app messaging. If your SDK version exposes a four-argument overload (`app`, `context`, `activity`, `json`), pass `applicationContext` as the second argument.

Step 6 — Manifest

XML

```
<application
    android:name=".MyApplication"
    android:icon="@mipmap/ic_launcher"
    android:label="@string/app_name"
    android:theme="@style/Theme.YourApp">

    <activity
        android:name=".MainActivity"
        android:exported="true"
        android:launchMode="singleTop">
        <intent-filter>
            <action android:name="android.intent.action.MAIN" />
            <category android:name="android.intent.category.LAUNCHER" />
        </intent-filter>
    </activity>

    <service
        android:name=".MyNotificationService"
        android:exported="false" />
</application>
```

launchMode

Use `singleTop` or `singleTask` on the click Activity so `onNewIntent` runs when the app is already open.

Step 7 — MainActivity wiring

KOTLIN

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_main)

    BNotifyApp.handleIntent(applicationContext, intent)

    BNotifyApp.setOnTokenListener(object : BNotifyApp.OnTokenListener {
        override fun onNewToken(token: String?) {
            Log.d("BNotify_Token", "BNotify Token: $token")
        }
    })

    intent.getStringExtra("screen")?.let { screen ->
        if (screen.equals("Test", true)) {
            BNotifyApp.setActivityToOpenOnClick(TestActivity::class.java)
            startActivity(Intent(this, TestActivity::class.java))
            finish()
        }
    }
}

override fun onNewIntent(intent: Intent) {
    super.onNewIntent(intent)
    setIntent(intent)
    BNotifyApp.handleIntent(applicationContext, intent)
}
```

Call	Why
<code>handleIntent</code>	Permission path, click extras, service restart
<code>setOnTokenListener</code>	Receives the BNotify device token after registration
<code>setActivityToOpenOnClick</code>	Which Activity future taps should open
<code>onNewIntent</code>	Required when app is already in memory

Step 8 — Custom MessagingService

KOTLIN

```
class MyNotificationService : BNotifyMessagingService() {
    override fun activityToOpenOnClick(): Class<out Activity> {
        return MainActivity::class.java
    }

    override fun onMessageReceived(remoteMessage: NotificationModel?) {
        Log.i("BNotify", "title: ${remoteMessage?.title}")
        // Custom UI when auto-notifications are disabled
    }
}
```

Notification permission (Android 13+)

The SDK declares `POST_NOTIFICATIONS`. For better UX, also request from your Activity:

KOTLIN

```
if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU) {
    ActivityCompat.requestPermissions(
        this,
        arrayOf(Manifest.permission.POST_NOTIFICATIONS),
        1001
    )
}
```

Screen / deep routing

Send a `screen` field in the notification payload. The SDK places it in click Intent extras — **your app** decides navigation (`home` , `Test` , or any custom route).

In-app messaging

In-app campaigns appear inside the app (modal, card, top banner, fullscreen, and image). They can still show if the user denied system notification permission, as long as `initializeBase` has run and the device is online.

Verification

1. Install on a physical device with network access.
2. Confirm logs show the BNotify token callback.
3. Send a test push from the BNotify dashboard.
4. Tap the notification → Activity opens → click event tracked.
5. Bring the app to the foreground and confirm an in-app campaign appears if one is scheduled.

Troubleshooting

Symptom	Likely cause	Fix
Gradle cannot resolve artifact	Missing JitPack repo	Add <code>maven { url = uri("https://jitpack.io") }</code> to both management blocks
No <code>bnotify-config.json</code>	Wrong path	File must be <code>app/bnotify-config.json</code>
No tray notification on API 33+	Permission denied	Request <code>POST_NOTIFICATIONS</code>
Tap does nothing when app open	Missing <code>onNewIntent</code>	Forward intent + use <code>singleTop</code>
No device on dashboard	Bad credentials / network	Verify the config JSON and that the device is online

Need help?

See the [FAQ](#) or [contact support](#) at bnotify93@gmail.com.

iOS SDK integration

Add BNotify with Swift Package Manager, enable Push Notifications, drop in `bnotify-config.plist`, and wire AppDelegate. Optionally add a Notification Service Extension for delivery tracking and rich images.

Create the app in Console first

Before SPM setup, [create a project](#), add an iOS app, and download `bnotify-config.plist`. Then continue below.

Distribution [SPM](#)

Minimum [iOS 13.0](#)

Product [BNotify](#)

Repo [GitLab](#)

What you get after integration

Capability	How it works
Permission prompt	SDK requests alert, sound, and badge
Device token	Forward the system token to the SDK at launch
Click / dismiss / action	Call <code>trackEvent</code> from the notification response
Delivery (<code>received</code>)	Optional Notification Service Extension
In-app messages	Modal, card, banner, fullscreen, and image
Default category	<code>bnotify</code> with action <code>OPEN</code>

Dashboard delivers campaigns

The iOS SDK does not send notifications. Register the device, then create and send campaigns from the BNotify dashboard. Upload your Apple Push credentials in the dashboard so iOS devices can receive them.

Prerequisites

Requirement	Notes
Xcode 15+	SPM + Swift package
Deployment Target $\geq$ 13.0	Matches SDK
Apple Developer Program	Required for real-device push
Push Notifications capability	App ID + profiles
Physical device	Required for real push testing
Dashboard plist	Download <code>bnotify-config.plist</code> after you register the iOS app in Console

Integration checklist

- 1 Add SPM product `BNotify` to the app target
- 2 Enable **Push Notifications** capability
- 3 Add `bnotify-config.plist` to the app target
- 4 Wire register APIs at launch
- 5 Forward the device token / registration error to the SDK
- 6 Implement click/dismiss `trackEvent`
- 7 App Groups + Notification Service Extension
- 8 Disable verbose logs in Release

Step 1 — Add the Swift Package

1. Xcode → **File** → **Add Package Dependencies...**
2. Enter: `https://gitlab.com/hamza.hafeez/bnotifyiossdk.git`
3. Choose version rule (e.g. Up to Next Major).
4. Add product **BNotify** to the main app (and NSE if you create one).

```
SWIFT
```

```
import BNotify
```

Step 2 — Apple Developer / Xcode capabilities

1. Apple Developer → Identifiers → enable **Push Notifications** on your App ID.
2. Renew provisioning profiles that include Push.
3. Upload your Apple Push key (`.p8`) or certificates in the **BNotify dashboard**.
4. In Xcode target → Signing & Capabilities → + **Capability** → **Push Notifications**.

For NSE delivery tracking, add the same **App Group** on the app and extension targets, then set `appGroupId` in the plist.

Step 3 — Add `bnotify-config.plist`

Download from the BNotify Console after you register the iOS app (see [Create a project](#)). Keep the filename exactly `bnotify-config.plist` and include it in the **main app** target membership. The SDK loads it from `Bundle.main`.

XML

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN" "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0">
  <dict>
    <key>projectId</key>
    <string>your-project-id</string>
    <key>packageName</key>
    <string>com.example.yourapp</string>
    <key>apiKey</key>
    <string>your-api-key</string>
    <key>appId</key>
    <string>your-app-id</string>
    <key>authDomain</key><string></string>
    <key>databaseURL</key><string></string>
    <key>storageBucket</key><string></string>
    <key>messagingSenderId</key><string></string>
    <key>measurementId</key><string></string>
  </dict>
</plist>
```

Config file

The dashboard download is enough. Do not add extra URL keys to the plist.

Step 4 — Wire AppDelegate

SWIFT

```
import UIKit
import UserNotifications
import BNotify

@main
class AppDelegate: UIResponder, UIApplicationDelegate, UNUserNotificationCenterDelegate {

    func application(
        _ application: UIApplication,
        didFinishLaunchingWithOptions launchOptions: [UIApplication.LaunchOptionsKey: Any]?
    ) -> Bool {
        UNUserNotificationCenter.current().delegate = self

        #if !DEBUG
        BNotifyLog.isEnabled = false
        #endif

        BNotifyManager.shared.registerCategories()
        BNotifyManager.shared.registerForPushNotifications()
        return true
    }

    func application(
        _ application: UIApplication,
        didRegisterForRemoteNotificationsWithDeviceToken deviceToken: Data
    ) {
        BNotifyManager.shared.didRegisterForRemoteNotifications(token: deviceToken)
    }

    func application(
        _ application: UIApplication,
        didFailToRegisterForRemoteNotificationsWithError error: Error
    ) {
        BNotifyManager.shared.didFailToRegisterForRemoteNotifications(error: error)
    }
}
```

Keep device-token callbacks on `AppDelegate` even when using `SceneDelegate`.

Step 5 — Track clicks, dismissals & actions

SWIFT

```
func userNotificationCenter(
    _ center: UNUserNotificationCenter,
    didReceive response: UNNotificationResponse,
    withCompletionHandler completionHandler: @escaping () -> Void
) {
    let userInfo = response.notification.request.content.userInfo
    let type: String
    let actionId: String

    switch response.actionIdentifier {
    case UNNotificationDismissActionIdentifier:
        type = "dismissed"; actionId = "dismiss"
    case UNNotificationDefaultActionIdentifier:
        type = "clicked"; actionId = "clicked"
    default:
        type = "action"; actionId = response.actionIdentifier
    }

    BNotifier.shared.trackEvent(type: type, userInfo: userInfo, actionId: actionId)
    // Optional: navigate from userInfo
    completionHandler()
}
```

User action	Suggested type	Suggested actionId
Tap body	clicked	clicked
Dismiss	dismissed	dismiss
Tap Open	action	OPEN

Step 6 — Foreground display (optional)

SWIFT

```
func userNotificationCenter(
    _ center: UNUserNotificationCenter,
    willPresent notification: UNNotification,
    withCompletionHandler completionHandler: @escaping (UNNotificationPresentationOptions) -> Void
) {
    if #available(iOS 14.0, *) {
        completionHandler([.banner, .sound, .badge])
    } else {
        completionHandler([.alert, .sound, .badge])
    }
}
```

Step 7 — Notification Service Extension (recommended)

Use an NSE to report `received` and attach images for collapsed thumbnail + expanded media (GIF animates natively).

1. File → New → Target → **Notification Service Extension**.
2. Link product **BNotify** to the extension.
3. Enable the same App Group on app + extension; set `appGroupId` in plist.

SWIFT

```
import UserNotifications
import BNotify

class NotificationService: UNNotificationServiceExtension {
    var contentHandler: ((UNNotificationContent) -> Void)?
    var bestAttemptContent: UNMutableNotificationContent?

    override func didReceive(
        _ request: UNNotificationRequest,
        withContentHandler contentHandler: @escaping (UNNotificationContent) -> Void
    ) {
        self.contentHandler = contentHandler
        bestAttemptContent = (request.content.mutableCopy() as? UNMutableNotificationContent)
        BNotifyExtensionSafe.prepareNotificationContent(request) { content in
            contentHandler(content)
        }
    }

    override func serviceExtensionTimeWillExpire() {
        if let contentHandler, let bestAttemptContent {
            contentHandler(bestAttemptContent)
        }
    }
}
```

In-app messages

In-app campaigns appear inside the app (modal, card, top banner, fullscreen, and image). They are separate from lock-screen notifications. Optional custom triggers:

SWIFT

```
BNotifyInApp.start() // optional; also starts from config load
BNotifyInApp.logEvent("purchase_completed")
```

Rich media payload

For NSE + rich media, include `mutable-content: 1` and put `imageUrl` as a sibling of `aps`:

JSON

```
{
  "aps": {
    "alert": { "title": "Hello", "body": "World" },
    "sound": "default",
    "mutable-content": 1,
    "category": "bnotify"
  },
  "notificationId": "n-123",
  "token": "<device-or-tracking-token>",
  "imageUrl": "https://cdn.example.com/promo.gif",
  "isExpanded": true
}
```

Without mutable-content

If `mutable-content` is missing, the NSE may not run — no `received` tracking and no expanded image attachment.

Verification

1. Run on a physical device with Push capability.
2. Grant notification permission; confirm the device appears in the BNotify dashboard.
3. Send a test push from the BNotify dashboard with `notificationId` and category `bnotify`.
4. Tap / dismiss → events appear in analytics.
5. With NSE: confirm `received` + image thumbnail / expand.

Troubleshooting

Symptom	Likely cause	Fix
No token / registration fails	Push capability or provisioning	Enable Push on App ID; use real device
Text-only push, no image	Missing NSE / <code>mutable-content</code> / <code>imageUrl</code>	Add NSE; put <code>imageUrl</code> outside <code>aps</code>
App not found on register	Wrong <code>appId</code> / bundle	Match dashboard app + <code>packageName</code>
No dismiss events	Category not <code>bnotify</code>	Call <code>registerCategories()</code> ; set category in payload
In-app never shows	Offline or incomplete init	Confirm config load, network, and that <code>registerForPushNotifications()</code> ran

Need help?

See the [FAQ](#) or [contact support](mailto:bnotify93@gmail.com) at bnotify93@gmail.com.

Flutter SDK integration

Add BNotify to any Flutter app with the `bnotify_flutter` package.

After setup, devices register automatically and can receive pushes plus in-app campaigns on Android and iOS. Web is not supported.

Create the apps in Console first

Before Dart setup, [create a project](#), [register Android and iOS apps](#), and download `bnotify-config.json` plus `bnotify-config.plist`. Package name and bundle id must match the Flutter app.

Package `bnotify_flutter`

Version 0.1.2

minSdk 24

iOS 13+

pub.dev `bnotify_flutter`

What you get after integration

Capability	Result
Device registration	Device appears in the BNotify dashboard after <code>initialize</code> + permission
Push notifications	System notifications on Android and iOS
Background / killed app	Notifications can still arrive after the first successful init
Notification tap	Dart <code>BNotify.onClick</code> with a <code>screen</code> value you can route on
In-app messages	Native in-app UI when Console campaigns match events you log
Device token	Dart <code>BNotify.onToken</code>

Dashboard delivers campaigns

The Flutter SDK does not send campaigns. Create and send messages from the BNotify dashboard. The SDK registers the device, displays messages, and reports engagement.

Prerequisites

Requirement	Notes
Flutter 3.24+ / Dart 3.13+	Matches the plugin <code>pubspec.yaml</code>
Android + iOS only	No web implementation
BNotify Console account	Project + Android app + iOS app
Android <code>minSdk</code> 24	Required
iOS 13.0+	Required
Physical devices	Recommended for push testing

Integration checklist

- 1 Register Android and iOS apps in Console with the exact `applicationId` / bundle id
- 2 Download `bnotify-config.json` and `bnotify-config.plist`
- 3 Add `bnotify_flutter: ^0.1.2` and run `flutter pub get`
- 4 Place JSON as a Flutter asset and at `android/app/bnotify-config.json`
- 5 Android: JitPack, minSdk 24, `BNotifyFlutterActivity`, `singleTask`, AppCompatActivity themes
- 6 iOS: plist on Runner, Push capability, optional Notification Service Extension
- 7 Dart: subscribe to streams, `initialize`, then `requestPermission`
- 8 Send a test campaign from the Console

Step 1 — Console

Open the [BNotify Console](https://console.bnotify.com/) (<https://console.bnotify.com/>). Create a project, then register:

- An **Android** app whose package name equals `applicationId` in `android/app/build.gradle.kts`
- An **iOS** app whose bundle id equals the Runner bundle identifier

Download `bnotify-config.json` and `bnotify-config.plist`. Do not commit real keys to a public repository. Full walkthrough: [Create a project](#).

Step 2 — Add the package

Install from [pub.dev](https://pub.dev/packages/bnotify_flutter) (https://pub.dev/packages/bnotify_flutter).

YAML

```
dependencies:
  bnotify_flutter: ^0.1.2

flutter:
  assets:
    - assets/bnotify-config.json
```

Then run `flutter pub get`. Copy the Console files to:

- `assets/bnotify-config.json`
- `android/app/bnotify-config.json` (same JSON — required so Android can start when a notification creates the process)
- `ios/Runner/bnotify-config.plist`

Install from pub.dev only. The plugin source repository is private (not open source).

Step 3 — Dart initialize, permission, and listeners

Subscribe to streams **before** `initialize` so you do not miss a token or a cold-start tap.

DART

```
import 'package:bnotify_flutter/bnotify_flutter.dart';
import 'package:flutter/material.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();

  BNotify.onToken.listen((token) {
    debugPrint('BNotify token: $token');
  });
  BNotify.onMessage.listen((message) {
    debugPrint('BNotify message: ${message.title} ${message.body}');
  });
  BNotify.onClick.listen((click) {
    debugPrint('BNotify click screen=${click.screen}');
  });

  await BNotify.initialize(
    configAsset: 'assets/bnotify-config.json',
  );

  runApp(const MyApp());
}
```

After the first frame (or from a settings screen), request notification permission:

DART

```
await BNotify.requestPermission();
```

API	When to call
<code>BNotify.initialize</code>	Once per process, after <code>ensureInitialized</code>
<code>BNotify.requestPermission</code>	Android 13+ / iOS notification permission
<code>BNotify.logEvent('...')</code>	Match an in-app campaign trigger
<code>BNotify.onToken</code>	Device token stream
<code>BNotify.onMessage</code>	Incoming payload (optional if system UI is on)
<code>BNotify.onClick</code>	Route after a notification tap
<code>BNotify.setAutoNotifications</code>	Android only: system tray vs custom UI

`configAsset` is read on Android. iOS uses `ios/Runner/bnotify-config.plist`. Pass `configAsset` anyway so one `initialize` call works on both platforms.

Hot reload does not re-run native Kotlin/Swift. After changing `MainActivity`, manifests, or the plist, do a full restart.

Step 4 — Android host setup

Do every item in this section. Skipping the Activity or theme steps is the usual reason in-app messages never appear.

JitPack + minSdk 24

Add JitPack in `android/settings.gradle.kts` (`pluginManagement.repositories`) and in `android/build.gradle.kts` repositories:

KOTLIN

```
maven { url = uri("https://jitpack.io") }
```

KOTLIN

```
defaultConfig {  
    applicationId = "com.example.my_app" // must match Console  
    minSdk = 24  
}
```

Copy config into Android assets

Keep `android/app/bnotify-config.json`. Add this above the `android { }` block in `android/app/build.gradle.kts`:

KOTLIN

```
val bnotifyConfigFile = file("bnotify-config.json")  
val bnotifyAssetsDir = file("src/main/assets")  
  
tasks.register<Copy>("copyBnotifyConfigAsset") {  
    from(bnotifyConfigFile)  
    into(bnotifyAssetsDir)  
    onlyIf { bnotifyConfigFile.exists() }  
}  
  
tasks.matching { it.name == "preBuild" }.configureEach {  
    dependsOn("copyBnotifyConfigAsset")  
}
```

MainActivity must extend `BNotifyFlutterActivity`

Do not extend `FlutterActivity` or `FlutterFragmentActivity`.

KOTLIN

```
import com.convex.bnotify_flutter.BNotifyFlutterActivity  
  
class MainActivity : BNotifyFlutterActivity()
```

Manifest and themes

Use `android:launchMode="singleTask"`. Do **not** set `android:taskAffinity=""` (that creates two Recents cards). `LaunchTheme` and `NormalTheme` must use `Theme.AppCompat.*` (for example `Theme.AppCompat.Light.NoActionBar`).

Call `BNotify.requestPermission()` at runtime on Android 13+. Keep the `INTERNET` permission.

Step 5 — iOS host setup

1. Copy `bnotify-config.plist` to `ios/Runner/bnotify-config.plist` and enable **Target Membership** → **Runner**.
2. In Xcode → Signing & Capabilities, add **Push Notifications**.
3. Minimum iOS 13. If you use Flutter's Swift Package Manager path: `flutter config --enable-swift-package-manager`.

A stock `FlutterAppDelegate` is enough. You do not need to call native BNotify APIs from `AppDelegate` for basic push.

Notification Service Extension (images)

To show images in the banner, add a Notification Service Extension, add the iOS SDK (`bnotifyiossdk` (<https://gitlab.com/hamza.hafeez/bnotifyiossdk.git>), product `BNotify`), enable the same App Group on Runner and the extension, then call:

SWIFT

```
BNotifyExtensionSafe.prepareNotificationContent(  
    request,  
    appId: bnotifyAppGroupId  
) { content in  
    contentHandler(content)  
}
```

Without an extension, text notifications still work. See the [iOS NSE section](#) for the full native pattern.

In-app campaigns

In the Console, create an in-app campaign whose trigger name matches a string you log (for example `app_open`):

DART

```
await BNotify.logEvent('app_open');
```

The name must match the Console trigger exactly. Android in-app UI requires `BNotifyFlutterActivity` and AppCompatActivity themes.

Notification tap routing

Listen to `BNotify.onClick` and use `click.screen` (set on the campaign in the Console) to navigate. Subscribe in `main()` before `initialize` so a tap that opened a killed app still reaches Dart.

DART

```
BNotify.onClick.listen((click) {  
  final screen = click.screen;  
  if (screen == null || screen.isEmpty) return;  
  navigatorKey.currentState?.pushNamed(screen);  
});
```

Optional: `await BNotify.setAutoNotifications(false)` on Android to draw your own UI from `onMessage`. iOS always uses the system notification UI.

Verification

Use a physical device and real Console files (not placeholders).

#	Test	Expected
1	Cold start, <code>initialize</code>	No exception; <code>onToken</code> prints a token
2	<code>requestPermission</code>	System dialog; user allows
3	Console device list	This install appears
4	Send a push, app backgrounded or swiped away	Tray / banner notification
5	Tap the notification	<code>onClick</code> fires; one Recents card
6	<code>logEvent</code> with a matching campaign	In-app UI appears

Troubleshooting

Symptom	What to check
<code>initialize</code> throws missing config	Asset listed in <code>pubspec.yaml</code> ; iOS plist on the Runner target
No token	Permission denied; package/bundle $\neq$ Console; placeholder keys
Pushes only while the UI is open	Android JSON not packaged as an asset; first init never succeeded
Two Recents cards	Remove empty <code>taskAffinity</code> ; use <code>singleTask</code>
In-app never shows (Android)	<code>MainActivity</code> still extends <code>FlutterActivity</code> ; themes not AppCompatActivity; full rebuild
Launch crash (Android theme)	<code>LaunchTheme</code> / <code>NormalTheme</code> must use <code>Theme.AppCompat.*</code>
Gradle cannot find <code>bnotifysdk</code>	JitPack missing from Gradle repositories
<code>minSdk</code> error	Set <code>minSdk = 24</code> in the app module
iOS images missing	No Notification Service Extension / App Group

Production

Pin `bnotify_flutter: ^0.1.2` , keep Console files out of public git, test `onClick` from a killed app, and complete APNs / Play signing as usual.

Send notifications from your backend

Use this HTTP API when your own server should send a BNotify notification — for example after an order, a payment, or any event in your backend. The mobile SDK only registers the device and displays the message. Your server decides when to send.

Copy the URL and key from Console

Open [BNotify Console \(https://console.bnotify.com/\)](https://console.bnotify.com/) → **Settings**. Use **Webhooks** as the request URL and **API Key** as `apiToken` in the JSON body. Do not hard-code another project's key.

Method POST**Body** JSON**Auth** `apiToken` in body**Target** `tokens` array

When to use this API

You want to...	Use
Compose and send from the BNotify dashboard	Console campaigns (no code)
Send from your server when something happens in your product	This Backend API
Register a phone and show the notification	Android , iOS , or Flutter SDK

Typical flow: the app SDK returns a device token → your app uploads that token to *your* server → your server POSTs this API with those tokens when it is time to notify.

Where to get the URL and API key

In the Console, open the project, then **Settings**.

Console field	Use it as
Webhooks	The POST URL. Example shape: <code>https://bnotify.convexinteractive.com/webhook</code> . Always copy the value shown for your project.
API Key	The <code>apiToken</code> field inside the JSON body. This identifies the project.

Keep the API key on the server

Call this endpoint only from your backend. Do not put the API key in a mobile app, a website frontend, or a public repository.

Endpoint

HTTP

```
POST {Webhooks URL from Console → Settings}
Content-Type: application/json
```

The current production webhook is usually:

URL

```
https://bnotify.convexinteractive.com/webhook
```

If Settings shows a different Webhooks value, use that value.

Request fields

Send one JSON object. `apiToken` and `tokens` are required for a targeted send. Include `scheduledAt` only when `isScheduled` is `true` .

Field	Type	Required	Description
<code>apiToken</code>	string	Yes	Project API Key from Console → Settings.
<code>tokens</code>	string[]	Yes	Device tokens that should receive this notification. One string per device.
<code>title</code>	string	Yes	Notification title.
<code>message</code>	string	Yes	Notification body.
<code>type</code>	string	No	Your label for the message, for example <code>promo</code> .
<code>screen</code>	string	No	Screen name the app should open on tap. The SDK exposes this on the click callback.
<code>action</code>	string	No	Action hint for the app, for example <code>navigate</code> .
<code>imageUrl</code>	string	No	HTTPS image URL. iOS expanded images also need a Notification Service Extension.
<code>deepLink</code>	string	No	Link the app can open, for example <code>convexinteractive.com</code> .
<code>isExpanded</code>	string	No	<code>"true"</code> or <code>"false"</code> . Controls expanded notification content.
<code>isScheduled</code>	boolean	Yes	<code>true</code> to send later. <code>false</code> to send immediately.
<code>scheduledAt</code>	string	Only if scheduled	Format <code>YYYY-MM-DD HH:mm:ss</code> , for example <code>2026-09-22 18:00:00</code> . Omit this field when <code>isScheduled</code> is <code>false</code> .

Send now vs schedule

Send immediately

Set `isScheduled` to `false` and **do not** send `scheduledAt` .

JSON

```
{
  "title": "Order shipped",
  "message": "Your order is on the way.",
  "type": "promo",
  "screen": "home",
  "action": "navigate",
  "imageUrl": "https://example.com/image.jpg",
  "apiToken": "YOUR_API_KEY",
  "isScheduled": false,
  "isExpanded": "false",
  "deepLink": "example.com",
  "tokens": [
    "DEVICE_TOKEN_1",
    "DEVICE_TOKEN_2"
  ]
}
```

Send at a specific time

Set `isScheduled` to `true` and include `scheduledAt`.

JSON

```
{
  "title": "Test Notification",
  "message": "This is a test notification to check system",
  "type": "promo",
  "screen": "home",
  "action": "navigate",
  "imageUrl": "https://example.com/image.jpg",
  "apiToken": "YOUR_API_KEY",
  "isScheduled": true,
  "isExpanded": "false",
  "deepLink": "example.com",
  "scheduledAt": "2026-09-22 18:00:00",
  "tokens": [
    "DEVICE_TOKEN_1"
  ]
}
```

Which devices receive it

`tokens` is the list of BNotify device tokens. The notification is sent only to those devices.

- Android / iOS / Flutter SDKs expose the token after registration (`onToken` in Flutter, the native token callback on Android and iOS).
- Store that token on your server, next to the user account.
- When you notify one user, send that user's token. When you notify many users, send every token in the same `tokens` array.

An empty `tokens` array does not identify a device. Always pass the tokens you collected from the SDK.

Examples

Replace `YOUR_API_KEY` with Settings → API Key, and replace device tokens with tokens from your database. The samples below send immediately (`isScheduled: false` , no `scheduledAt`).

cURL

BASH

```
curl --location 'https://bnotify.convexinteractive.com/webhook' \  
  --header 'Content-Type: application/json' \  
  --data '{  
    "title": "Test Notification",  
    "message": "This is a test notification to check system",  
    "type": "promo",  
    "screen": "home",  
    "action": "navigate",  
    "imageUrl": "https://example.com/image.jpg",  
    "apiToken": "YOUR_API_KEY",  
    "isScheduled": false,  
    "isExpanded": "false",  
    "deepLink": "example.com",  
    "tokens": [  
      "DEVICE_TOKEN_1"  
    ]  
  }'
```

Node.js

JAVASCRIPT

```
const response = await fetch("https://bnotify.convexinteractive.com/webhook", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({
    title: "Test Notification",
    message: "This is a test notification to check system",
    type: "promo",
    screen: "home",
    action: "navigate",
    imageUrl: "https://example.com/image.jpg",
    apiToken: process.env.BNOTIFY_API_KEY,
    isScheduled: false,
    isExpanded: "false",
    deepLink: "example.com",
    tokens: ["DEVICE_TOKEN_1"],
  }),
});

if (!response.ok) {
  throw new Error(`BNotify webhook failed: ${response.status}`);
}
```

Python

PYTHON

```
import os
import requests

payload = {
    "title": "Test Notification",
    "message": "This is a test notification to check system",
    "type": "promo",
    "screen": "home",
    "action": "navigate",
    "imageUrl": "https://example.com/image.jpg",
    "apiToken": os.environ["BNOTIFY_API_KEY"],
    "isScheduled": False,
    "isExpanded": "false",
    "deeplink": "example.com",
    "tokens": ["DEVICE_TOKEN_1"],
}

response = requests.post(
    "https://bnotify.convexinteractive.com/webhook",
    json=payload,
    timeout=30,
)
response.raise_for_status()
```

PHP

PHP

```
$payload = [
    "title" => "Test Notification",
    "message" => "This is a test notification to check system",
    "type" => "promo",
    "screen" => "home",
    "action" => "navigate",
    "imageUrl" => "https://example.com/image.jpg",
    "apiToken" => getenv("BNOTIFY_API_KEY"),
    "isScheduled" => false,
    "isExpanded" => "false",
    "deepLink" => "example.com",
    "tokens" => ["DEVICE_TOKEN_1"],
];

$ch = curl_init("https://bnotify.convexinteractive.com/webhook");
curl_setopt_array($ch, [
    CURLOPT_POST => true,
    CURLOPT_HTTPHEADER => ["Content-Type: application/json"],
    CURLOPT_POSTFIELDS => json_encode($payload),
    CURLOPT_RETURNTRANSFER => true,
]);
$body = curl_exec($ch);
$status = curl_getinfo($ch, CURLINFO_HTTP_CODE);
curl_close($ch);

if ($status < 200 || $status >= 300) {
    throw new RuntimeException("BNotify webhook failed: " . $status);
}
```

Integration checklist

- 1 Create the project and register the mobile app in the [Console](#)
- 2 Integrate the [Android](#), [iOS](#), or [Flutter](#) SDK so devices produce a token
- 3 Save each device token on your server
- 4 Copy **Webhooks** and **API Key** from Console → Settings

- 5 POST JSON from your backend with `apiToken` and `tokens`
- 6 For an immediate send, set `isScheduled` to `false` and omit `scheduledAt`
- 7 For a later send, set `isScheduled` to `true` and set `scheduledAt` to `YYYY-MM-DD HH:mm:ss`
- 8 Confirm the notification on a physical device that owns one of the tokens

Security

- Store the API key in a server secret (environment variable or secret manager), not in source control.
- Call the webhook only from your backend.
- Send tokens only for the users who should see that message.
- If an API key is exposed, generate a new one in the Console and update your server.

Tap routing

Set `screen` to the same route name your app reads from the SDK click callback (`onClick` / notification extras). The API delivers the payload; the app decides which screen to open.

Web SDK

COMING SOON

Browser push for websites is on the BNotify roadmap. You can already see **Web** as a platform option in the Console when you add an app. A public Web SDK and integration guide will ship here when it is ready.

Not available yet

Do not wait on the Web SDK for production apps today. Use [Android](#) or [iOS](#), or [contact us](#) for early access.

Meanwhile

- Create a project in the [BNotify Console](#) for your mobile apps.
- Ask for Web SDK updates via [Contact](#) (bnotify93@gmail.com).

React Native SDK COMING SOON

A React Native package is planned. It will expose a JavaScript API so you can initialize BNotify and handle notifications from your RN app.

Not available yet

No npm package is published yet. For production, integrate native [Android](#) / [iOS](#) SDKs, or request early access via [Contact](#).

Planned integration shape

Step	Expectation
Dependency	<code>npm i @bnotify/react-native</code> (name TBD)
Autolinking	Android Gradle + iOS CocoaPods / SPM bridge
Config	Same JSON / plist credentials as native SDKs
JS API	Init, token events, notification listeners, in-app hooks

Meanwhile

- Follow native guides for Android and iOS modules inside your RN project.
- Email bnotify93@gmail.com to join the React Native waitlist.

Frequently asked questions

Short answers to the questions teams ask most when integrating BNotify.
For anything not covered here, use the [contact form](#).

– Does the SDK send push notifications?

No. The SDK registers the device and displays / tracks messages. You create and send campaigns from the BNotify dashboard, or POST to the [Backend API](#) from your own server.

+ Which platforms are available today?

+ How do I create a project and add an app?

+ Where do I get credentials?

+ What is the Android Maven coordinate?

+ What is the iOS package URL?

+ Why don't notifications appear on Android 13+?

+ Why is my iOS push text-only with no image?

+ Are in-app messages the same as pushes?

+ Can I use the SDKs offline / without a dashboard project?

+ How do I send a notification from my own backend?

+ What is the Flutter package name?

+ How do I get help for Web or React Native?

+ Is there a sample / demo app?

Still stuck?

Open [Contact](#) and include platform, SDK version, and a short log excerpt (redact API keys).

[← React Native · Contact →](#)

BNotify FAQ

BNotify SDK Documentation · <https://bnotify.github.io/sdk-docs/>

Contact

Questions about integration, credentials, or upcoming Web / React Native SDKs? Reach the BNotify team directly.



CONSOLE DASHBOARD

console.bnotify.com/projects

Open the BNotify Console to manage projects, apps, and campaigns.



SUPPORT EMAIL

bnotify93@gmail.com

Temporary contact — official domain email will replace this later.

Send a message

Submitting opens your email client with a pre-filled message to **bnotify93@gmail.com**.

Name

Work email

Platform



How can we help?

Describe your issue, include SDK version and short logs (redact API keys).

Open email to bnotify93@gmail.com

Before you write

- Check the [FAQ](#) and platform guide ([Android](#) / [iOS](#)).
- Include OS version, app package / bundle ID, and whether you tested on a physical device.
- Never paste full API keys or signing credentials in email.